Git & GitHub Notes

Version Control System (VCS)

Version control, also known as source control, is tracking and managing software code changes.

Introduction Git

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git Global Configuration

To set or check your global Git identity, use the following commands in your terminal or command prompt:

The `--global` flag ensures these settings apply to all repositories on your computer unless overridden by local repository settings.

- **Set Username:**
  `git config --global user.name "Your Name"`

- **Set Email:**
  `git config --global user.email "your.email@example.com"`

You can confirm your configuration using these commands:

- **Check Username:** `git config --global user.name`
- **Check Email:** `git config --global user.email`
- **List All Settings:** `git config --list` (shows all active configurations and their sources)

Initializing Git Project

Git `init` is a command used to initialize a new Git repository. It creates a hidden .git folder in your project directory, which allows Git to start tracking changes, manage versions, and support collaboration.

.Git Folder includes:

- **HEAD:** Points to the current branch (usually main).
- **config:** Stores repository settings like username, email, and Git preferences.
- **description:** Contains a short description of the repository (optional).
- **objects folder:** Stores Git data such as file versions, commits, and snapshots.
- **refs folder:** Holds references to branches and tags that point to commits

Using git Init

1. **Navigate to Your Project Directory**

   Use the terminal to move to the folder where you want the repository: `cd path/to/your/project\`

2. **Initialize the Repository**

   Run the following command: `git init`

Adding Files in Git Projects

Adding a file to Git is a multi-step process that involves moving the file from your local computer to a "staging area" before permanently recording it in the repository's history.

Use the `git add` command to tell Git you want to include specific changes in the next snapshot.

- To add a **single file**: `git add filename.txt`
- To add **all new and changed files**: `git add .` or `git add -A`

Git Commit

A commit is like a save point in your project. It records a snapshot of your files at a certain time, with a message describing what changed.

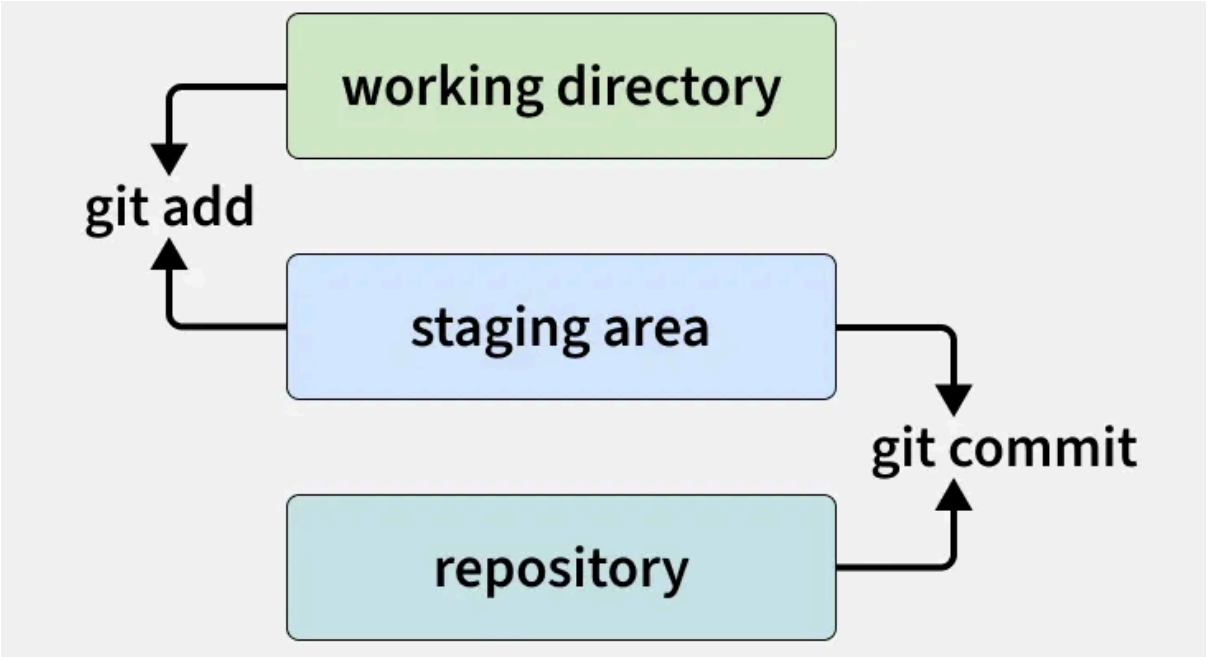
Here are some key commands for commits:

- `git commit -m "message"` - Commit staged changes with a message
- `git commit -a -m "message"` - Commit all tracked changes (skip staging)
- `git log` - See commit history

Staging Area

Git, the staging area (also called the index) is an intermediate space where changes are gathered before they are committed. Think of it as a draft board: you can organize and review what changes you want to include in the next commit.

The `git status` command displays the current state of the working directory and the staging area.



Git Revert and Reset

Git Revert is used to undo a change. This operation creates a new commit that undoes the change by reversing it. Revert is a safer option when working with other developers, as it preserves the commit history and doesn't affect others' work.

To perform a revert operation, use the following command:

```
git revert <commit-hash>
```

Git Reset is used to undo operations by rewriting the commit history. This operation removes all commits between the current HEAD and the specified commit, and it restores the files in the working directory to the state of the specified commit. Reset can cause problems when working with other developers, so it should be used with caution.

To perform a reset operation, use the following command:

```
git reset --hard <commit-hash>
```

GitHub

GitHub is a cloud-based platform used to host Git repositories, allowing developers to store their code online and collaborate with others globally. It enhances the core functionality of Git with a graphical user interface (GUI) and project management tools.

Git VS GitHub

Git	GitHub
A software tool.	A web-based service.
A distributed version control system (DVCS).	A repository hosting and collaboration platform.
Primarily used through a command-line interface.	Provides a web-based graphical interface.
Installed locally on a developer's system.	Hosted on the web.
Maintained by the open-source community.	Maintained by Microsoft.

Focuses on version control and change tracking.	Focuses on code hosting and collaboration.
Manages source code history.	Hosts version-controlled repositories.
First released in 2005.	Launched in 2008.
No built-in user access management	Includes user and access management.
Open-source licensed.	Offers free and paid plans.
Limited external tool integration.	Provides a marketplace for integrations.
Git competes with CVS, SVN, Mercurial, etc.	GitHub competes with GitLab, Bit Bucket, Azure DevOps Server, etc.

Git Remote

A remote repository is hosted on a remote (this could be on the internet or an off-site server; it could even be the same machine in a different path) and is shared among multiple team members. A local repository is hosted on a local machine for an individual user.

Common Commands

- **List remotes:** `git remote` lists the names (shortnames) of all connected remotes.
- **Show URLs:** `git remote -v` (verbose) shows the specific URLs for fetching and pushing alongside the names.
- **Add a remote:** `git remote add <name> <url>` establishes a new connection. "origin" is the standard default name used for your main remote.
- **Remove a remote:** `git remote remove <name>` unlinks a remote repository from your local setup.
- **Rename a remote:** `git remote rename <old-name> <new-name>` changes the shortname of a remote.
- **Update URL:** `git remote set-url <name> <new-url>` changes the address of an existing remote.
- **Show details:** `git remote show <name>` provides detailed information about a specific remote, including which branches it tracks.

Git - Origin

In Git, origin is the default name given to the remote repository where the project is hosted. It is commonly used with branch names to push or pull changes between local and remote repositories

Git Push

git push uploads commits from the local repository to a remote repository so that changes become available to other collaborators.

Syntax:

```
git push <remote> <branch>
```

Git Pull

Git pull is a command which is used to fetch and integrate the changes which are present in the remote repository to the local repository.

Syntax

```
git pull [remote-name] [branch-name]
```

Git Branch

In Git, a branch is a lightweight, movable pointer to a specific commit. It creates an independent line of development, allowing you to work on features or fixes in isolation without affecting the main project.

1. Creation & Switching

These commands allow you to start new work silos or move between existing ones.

- **Create a branch**: `git branch <branch-name>`.
- **Switch to a branch**: `git checkout <branch-name>` or the modern `git switch <branch-name>`.
- **Create and switch simultaneously**: `git checkout -b <branch-name>` or `git switch -c <branch-name>`.
- **Switch to previous branch**: `git switch -`.

2. Viewing & Listing

Use these to understand the current state of your repository's branches. [1]

- **List local branches**: `git branch` (the current branch is marked with an `*`).
- **List all branches (local + remote)**: `git branch -a`.
- **View last commit on each branch**: `git branch -v`.
- **Visualize branch history**: `git log --oneline --graph --all` (provides a text-based branch tree).

3. Integration & Cleanup

Once work is finished, branches are typically merged back and then removed. [1, 2]

- **Merge into current branch**: `git merge <branch-name>`.
- **Delete a merged branch**: `git branch -d <branch-name>`.
- **Force delete an unmerged branch**: `git branch -D <branch-name>`.
- **Rename a branch**: `git branch -m <old-name> <new-name>`.

4. Remote Branching

Commands for synchronizing your local branches with a remote server like GitHub or GitLab.

- **Push local branch to remote**: `git push origin <branch-name>`.
- **Set upstream (track remote)**: `git push -u origin <branch-name>`.
- **Delete a remote branch**: `git push origin --delete <branch-name>`